GLUTEN FREE LIFE

Pruebas del sistema

Etapa 3 · Secciones 5, 6 y 7: pruebas unitarias, pruebas de integración y alcance lógico

Override Software Solutions · «Del barrio, para el barrio» · Cliente: Gluten Free Life · Noviembre de 2026

5. Pruebas unitarias

Las pruebas unitarias verifican cada unidad mínima del código —aquí, funciones puras de la lógica de negocio y de validación— de manera aislada: no usan base de datos, red ni interfaz. Son automáticas y se pueden ejecutar tantas veces como se quiera, lo que permite detectar errores de forma temprana y modificar el sistema con mayor seguridad.

5.1 Herramienta y ejecución

Aspecto	Detalle
Framework de testing	Vitest 2.1.9 (TypeScript, entorno Node.js 22)
Ubicación del código	backend/src/test/unit/ (dominio.test.ts, pedidos.test.ts, validaciones.test.ts)
Configuración	backend/vitest.config.ts (incluye solo src/test/unit/**/*.test.ts)
Comando	cd backend && npm run test:unit
Requisitos	Solo las dependencias de Node (npm install). No requiere PostgreSQL.
Cantidad	25 casos de prueba en 3 archivos, agrupados en 7 bloques (describe)

Se seleccionaron las funciones que contienen reglas de negocio centrales del comercio (precio, stock crítico, vencimientos y reposición) y las validaciones de entrada que protegen los datos (venta e importación de catálogo). Cada caso responde a una regla documentada en el modelado (R1, R2, R6, R8) o a una validación del sistema.

5.2 Pruebas representativas

A continuación se documentan las pruebas, indicando qué se probó, el resultado esperado y el resultado obtenido en la ejecución real (ver evidencia en 5.3).

PU-1 · Cálculo del precio sugerido (regla R2)

Qué se probó	Función actualizarPrecioVenta(precioCosto, porcentajeMargen): precio de venta = costo × (1 + margen/100), redondeado al peso. Se prueba con un caso común, con redondeo, con margen 0 y con costo 0.
Resultado esperado	costo 1000, margen 40 % → 1400 costo 1234,5, margen 35 % → 1667 (1666,575 redondeado) costo 850, margen 0 % → 850 costo 0, margen 40 % → 0
Resultado obtenido	4 de 4 casos correctos. Los valores devueltos coinciden con los esperados.

```
export function actualizarPrecioVenta(precioCosto: number, porcentajeMargen: number): number {
  return Math.round(precioCosto * (1 + porcentajeMargen / 100));
}

describe('Producto.actualizarPrecioVenta (R2)', () => {
  it('aplica el margen sobre el costo', () => {
    expect(actualizarPrecioVenta(1000, 40)).toBe(1400);
  });
  it('redondea al peso más cercano', () => {
    // 1234,5 × 1,35 = 1666,575 → 1667
    expect(actualizarPrecioVenta(1234.5, 35)).toBe(1667);
  });
  it('con margen 0 el precio es el costo', () => {
    expect(actualizarPrecioVenta(850, 0)).toBe(850);
  });
  it('con costo 0 devuelve 0', () => {
    expect(actualizarPrecioVenta(0, 40)).toBe(0);
  });
});
```

PU-2 · Detección de stock crítico y estado de stock (regla R1)

Qué se probó	Funciones evaluarStockCritico y estadoStock. La regla indica que un producto es crítico cuando stock_actual ≤ stock_minimo. Se prueba especialmente el caso borde (stock igual al mínimo), que es donde suelen aparecer los errores.
Resultado esperado	stock 2, mínimo 5 → crítico stock 5, mínimo 5 (borde) → crítico stock 6, mínimo 5 → no crítico estadoStock con umbral 3: 5→CRITICO, 7→BAJO, 9→NORMAL
Resultado obtenido	4 de 4 casos correctos. Además se comprobó que la prueba detecta defectos: al cambiar a propósito «<=» por «<» en la regla, 2 casos fallaron (ver 5.3, figura 2); luego se restauró el código original.

```
export function evaluarStockCritico(p: { stock_actual: number; stock_minimo: number }): boolean {
  return p.stock_actual <= p.stock_minimo;
}

describe('Producto.evaluarStockCritico (R1)', () => {
  it('es crítico cuando el stock es menor al mínimo', () => {
    expect(evaluarStockCritico({ stock_actual: 2, stock_minimo: 5 })).toBe(true);
  });
  it('es crítico cuando el stock es IGUAL al mínimo (borde)', () => {
    expect(evaluarStockCritico({ stock_actual: 5, stock_minimo: 5 })).toBe(true);
  });
  it('no es crítico cuando supera el mínimo', () => {
    expect(evaluarStockCritico({ stock_actual: 6, stock_minimo: 5 })).toBe(false);
  });
  it('estadoStock distingue CRITICO, BAJO y NORMAL', () => {
    expect(estadoStock({ stock_actual: 5, stock_minimo: 5 }, 3)).toBe('CRITICO');
    expect(estadoStock({ stock_actual: 7, stock_minimo: 5 }, 3)).toBe('BAJO');
    expect(estadoStock({ stock_actual: 9, stock_minimo: 5 }, 3)).toBe('NORMAL');
  });
});
```

PU-3 · Caducidad de un vencimiento (regla R6)

Qué se probó	Función verificarCaducidad(fechaVencimiento, hoy). Un vencimiento está vencido solo si su fecha es anterior a hoy; si vence hoy todavía se puede vender.
Resultado esperado	2026-11-04 respecto de 2026-11-05 → vencido 2026-11-05 respecto de 2026-11-05 → no vencido (borde) 2026-12-01 respecto de 2026-11-05 → no vencido
Resultado obtenido	3 de 3 casos correctos.

```
export function verificarCaducidad(fechaVencimiento: string, hoy: string): boolean {
  return fechaVencimiento < hoy;
}

describe('VencimientoProducto.verificarCaducidad (R6)', () => {
  it('una fecha pasada está vencida', () => {
    expect(verificarCaducidad('2026-11-04', '2026-11-05')).toBe(true);
  });
  it('vencer hoy todavía NO es vencido (borde)', () => {
    expect(verificarCaducidad('2026-11-05', '2026-11-05')).toBe(false);
  });
  it('una fecha futura no está vencida', () => {
    expect(verificarCaducidad('2026-12-01', '2026-11-05')).toBe(false);
  });
});
```

PU-4 · Cantidad sugerida de reposición (regla R8)

Qué se probó	Función cantidadSugerida(stockMinimo, stockActual, tipoVenta): sugiere reponer hasta el doble del mínimo, nunca menos de 1; en productos por unidad redondea hacia arriba a un entero y en granel conserva hasta 3 decimales.
Resultado esperado	mínimo 10, stock 3 (unidad) → 17 mínimo 5, stock 40 (unidad) → 1 mínimo 2,5, stock 1 (unidad) → 4 mínimo 2,5, stock 1,2345 (granel) → 3,766
Resultado obtenido	4 de 4 casos correctos.

```
export function cantidadSugerida(stockMinimo: number, stockActual: number, tipoVenta: string):
number {
  const c = Math.max(stockMinimo * 2 - stockActual, 1);
  return tipoVenta === 'UNIDAD' ? Math.ceil(c) : round3(c);
}

describe('cantidadSugerida (R8)', () => {
  it('repone hasta el doble del mínimo', () => {
    expect(cantidadSugerida(10, 3, 'UNIDAD')).toBe(17);
  });
  it('nunca sugiere menos de 1', () => {
    expect(cantidadSugerida(5, 40, 'UNIDAD')).toBe(1);
  });
  it('en productos por unidad redondea hacia arriba a un entero', () => {
    expect(cantidadSugerida(2.5, 1, 'UNIDAD')).toBe(4);
  });
  it('en granel conserva hasta 3 decimales', () => {
    expect(cantidadSugerida(2.5, 1.2345, 'GRANEL')).toBe(3.766);
  });
});
```

PU-5 · Validación de los datos de una venta

Qué se probó	Esquema ventaSchema (Zod), usado por POST /api/ventas para validar el cuerpo de la petición antes de tocar la base de datos.
Resultado esperado	venta válida (efectivo, 1 producto) → aceptada venta sin productos → rechazada con «La venta no tiene productos» medio de pago inexistente → rechazada cantidad 0 o negativa → rechazada
Resultado obtenido	4 de 4 casos correctos; el mensaje de error coincide con el que ve el usuario.

```
describe('ventaSchema — validación de entrada', () => {
  const ok = { canal_pago: 'EFECTIVO', monto_recibido: 5000, items: [{ id_producto: 1, cantidad:
2 }] };

  it('acepta una venta válida', () => {
    expect(ventaSchema.safeParse(ok).success).toBe(true);
  });
  it('rechaza una venta sin productos', () => {
    const r = ventaSchema.safeParse({ ...ok, items: [] });
    expect(r.success).toBe(false);
    if (!r.success) expect(r.error.issues[0].message).toBe('La venta no tiene productos');
  });
  it('rechaza un medio de pago inexistente', () => {
    expect(ventaSchema.safeParse({ ...ok, canal_pago: 'BITCOIN' }).success).toBe(false);
  });
  it('rechaza cantidades en cero o negativas', () => {
    expect(ventaSchema.safeParse({ ...ok, items: [{ id_producto: 1, cantidad: 0 }]
}).success).toBe(false);
    expect(ventaSchema.safeParse({ ...ok, items: [{ id_producto: 1, cantidad: -1 }]
}).success).toBe(false);
  });
});
```

PU-6 · Normalización de datos de la importación de catálogo

Qué se probó	Funciones parseNum y parseModalidad, que interpretan lo que el comercio escribe a mano en Excel: números con formato argentino (8.500,50), con símbolo $ y sinónimos de modalidad (u, kg, a granel).
Resultado esperado	"8.500,50" → 8500,5 "$ 8500,5" → 8500,5 1200 → 1200 "" / undefined / "abc" → null "Unidad" → UNIDAD · "KG" → GRANEL · "a granel" → GRANEL · "caja" / "" → null
Resultado obtenido	6 de 6 casos correctos.

```
describe('importación — parseNum', () => {
  it('interpreta el formato argentino 8.500,50', () => {
    expect(parseNum('8.500,50')).toBe(8500.5);
  });
  it('interpreta coma decimal y símbolo $', () => {
    expect(parseNum('$ 8500,5')).toBe(8500.5);
  });
  it('acepta números ya numéricos', () => {
    expect(parseNum(1200)).toBe(1200);
  });
  it('devuelve null ante vacío o texto no numérico', () => {
    expect(parseNum('')).toBeNull();
    expect(parseNum(undefined)).toBeNull();
    expect(parseNum('abc')).toBeNull();
  });
});

describe('importación — parseModalidad', () => {
  it('reconoce sinónimos de UNIDAD y GRANEL sin importar mayúsculas', () => {
    expect(parseModalidad('Unidad')).toBe('UNIDAD');
    expect(parseModalidad('KG')).toBe('GRANEL');
    expect(parseModalidad('a granel')).toBe('GRANEL');
  });
  it('devuelve null si no es una modalidad válida', () => {
    expect(parseModalidad('caja')).toBeNull();
    expect(parseModalidad('')).toBeNull();
  });
});
```

5.3 Evidencia de ejecución

Ejecución de npm run test:unit: 3 archivos, 25 pruebas, todas correctas.

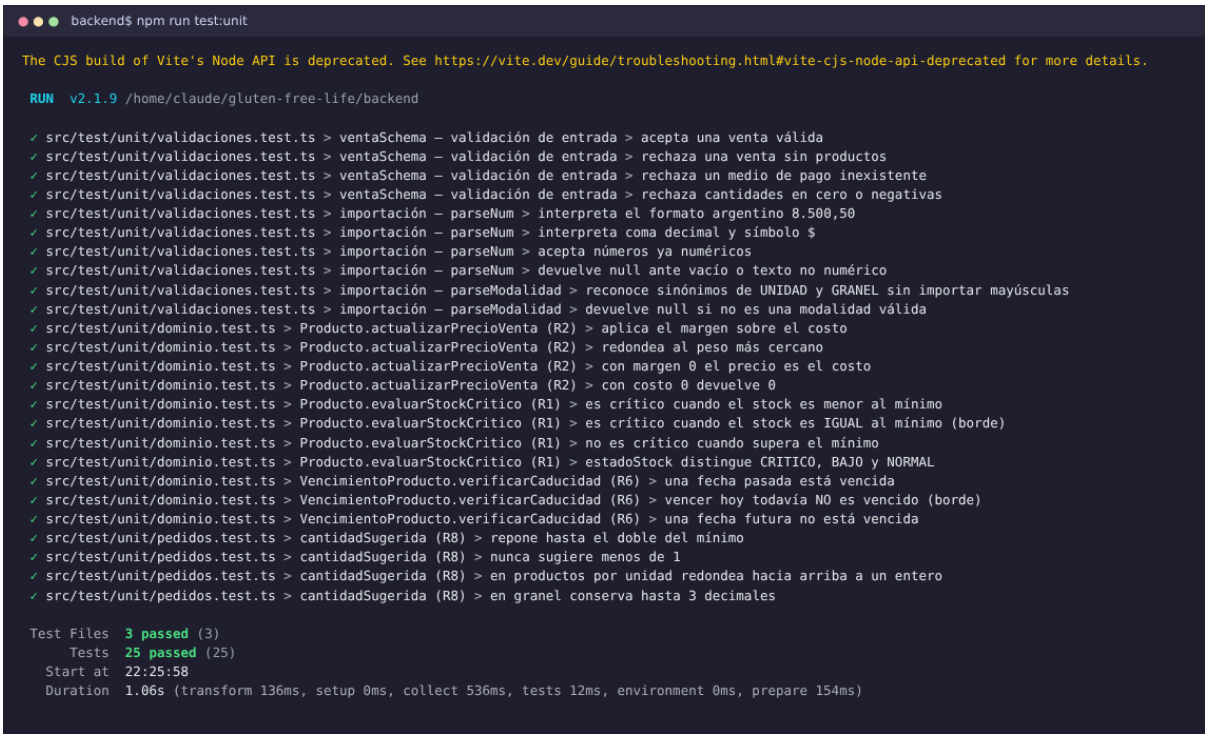


Figura 1. Ejecución de las pruebas unitarias con Vitest: 25 de 25 correctas.

Para demostrar que las pruebas realmente detectan errores, se alteró temporalmente la regla R1 (se reemplazó «<=» por «<»). Las pruebas del caso borde fallaron de inmediato y señalaron la línea y los valores esperado y recibido. Luego el código se restauró.



Figura 2. Con la regla R1 alterada a propósito, 2 pruebas fallan: la prueba detecta el defecto.

6. Pruebas de integración

Las pruebas de integración verifican que los módulos funcionen correctamente en conjunto: interfaz, API, lógica de negocio y base de datos PostgreSQL. A diferencia de las unitarias, aquí intervienen todos los componentes reales y el resultado se comprueba consultando directamente la base.

6.1 Entorno de prueba

Componente	Detalle
Base de datos	PostgreSQL 16 real, con el esquema de backend/migrations/001_init.sql y los datos de demostración (npm run seed)
Servidor	Node.js 22 + Express, versión compilada (node dist/index.js) en http://localhost:4000, sirviendo también el frontend compilado
Navegador	Chromium controlado con Playwright (resolución 1440 × 900)
Usuario	admin / 1234 (cuenta de demostración)
Scripts	docs/pruebas/integracion-venta.sh, docs/pruebas/ui-crud-productos.mjs y backend/src/test/e2e.ts

Código	Prueba	Módulos involucrados	Resultado
PI-1	Registrar una venta y comprobar su impacto en la base	API · servicio de ventas · reglas R1, R3, R4 · PostgreSQL	Correcta
PI-2	Alta, consulta, modificación y baja de un producto desde la interfaz	Interfaz React · API · validaciones · PostgreSQL	Correcta
PI-3	Suite de integración de la API (17 grupos)	API completa · servicios · PostgreSQL	17 de 17 correctos

6.2 PI-1 · Venta: de la API a la base de datos

PI-1 · Registrar una venta y comprobar su impacto en la base

Objetivo	Verificar que una venta enviada a la API se guarde en PostgreSQL y que la lógica de negocio actualice correctamente el stock y el vencimiento más próximo, y que una venta imposible sea rechazada sin alterar los datos.
Pasos realizados	1. Iniciar sesión con POST /api/auth/login y obtener el token. 2. Consultar en PostgreSQL el estado inicial de «Granola Sin TACC»: stock 9, mínimo 3, con dos vencimientos (4 u. al 21/11/2026 y 5 u. al 06/03/2027). 3. Enviar POST /api/ventas con 6 unidades de Granola, pago en efectivo, recibiendo $ 60.000. 4. Revisar la respuesta de la API. 5. Consultar nuevamente la base: producto, vencimientos, venta y detalle. 6. Enviar una segunda venta de 50 unidades (más que el stock) y comprobar que la base no cambia.
Resultado esperado	• HTTP 201, total $ 23.520 (6 × $ 3.920), vuelto $ 36.480 y venta REGISTRADA. • Aviso de stock crítico: el stock queda en 3, igual al mínimo (regla R1). • Stock del producto: 9 → 3. • Regla R4: se agota primero el vencimiento más próximo (4 u., que se elimina) y el resto se descuenta del siguiente (5 → 3). • Una fila en venta y una en detalle_venta con el precio histórico $ 3.920. • La venta de 50 unidades responde HTTP 409 «Stock insuficiente» y no modifica stock ni cantidad de ventas.

Resultado obtenido	• HTTP 201 · ticket 66 · total 23520 · recibido 60000 · vuelto 36480 · REGISTRADA. • alertas_stock_critico: Granola Sin TACC, stock 3, mínimo 3. • Base de datos: stock 3,000; el vencimiento del 21/11 fue consumido y eliminado; el del 06/03/2027 pasó de 5 a 3 unidades. • Base de datos: venta 66 con detalle de 6 u. a $ 3.920 (subtotal $ 23.520). • La venta de 50 unidades devolvió HTTP 409 «Stock insuficiente: Granola Sin TACC (disponible 3)»; el stock siguió en 3 y las ventas en 66. **Resultado: correcta.**

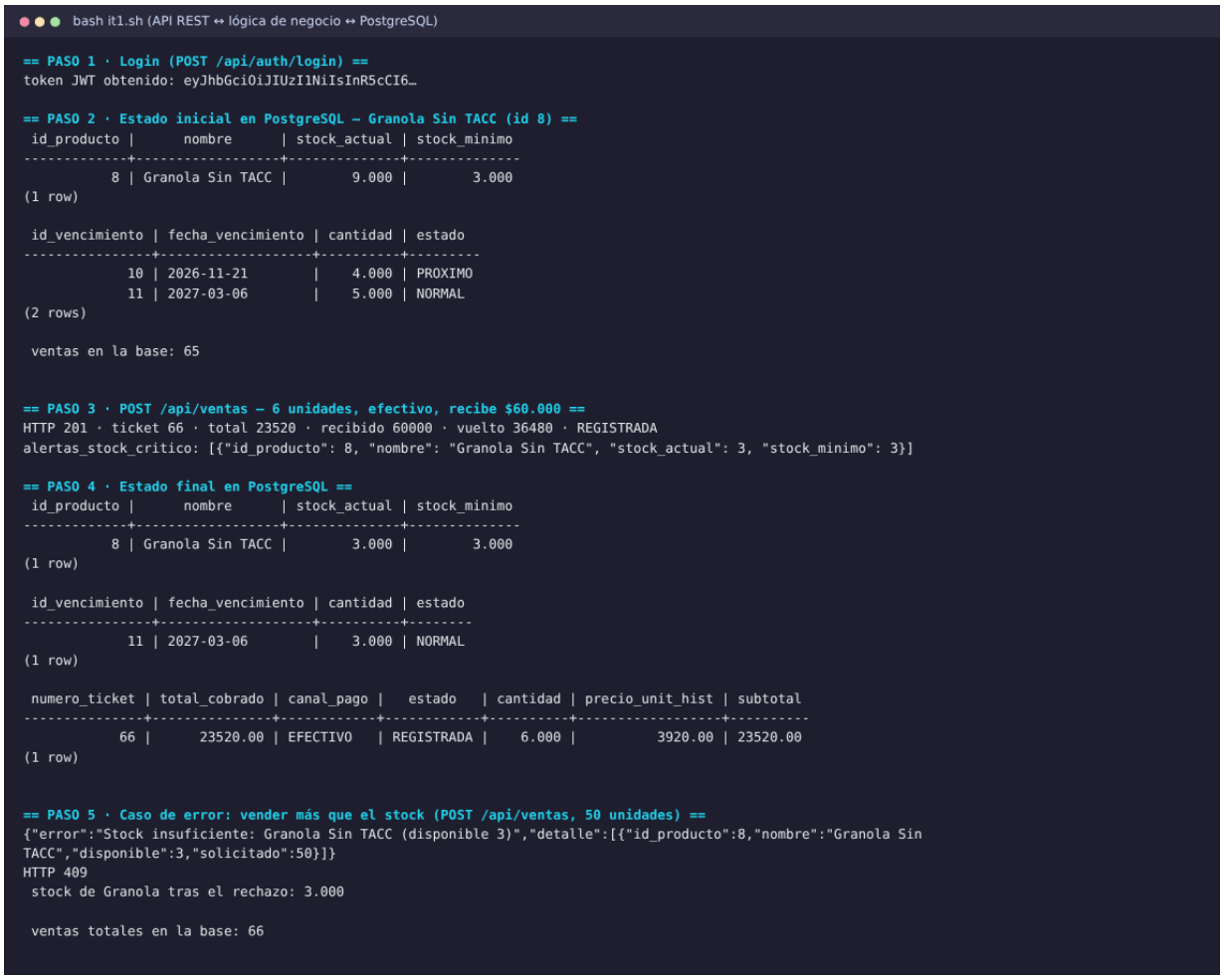


Figura 3. Registro de ejecución de PI-1: peticiones a la API y consultas SQL antes y después de la venta.

6.3 PI-2 · Producto: de la interfaz a la base de datos (CRUD)

PI-2 · Alta, consulta, modificación y baja de un producto desde la interfaz

Objetivo	Verificar que las cuatro operaciones básicas de persistencia (crear, leer, actualizar y eliminar) funcionen desde la pantalla Productos, que impacten en PostgreSQL y que las validaciones y la confirmación de baja actúen como corresponde.

Pasos realizados	1. Ingresar al sistema y abrir Productos → Nuevo producto. 2. **Crear:** completar Alfajor de maicena sin TACC, código 7790009999991, categoría Galletitas, unidad, stock mínimo 5, costo $ 800, margen 50 % y stock inicial 20; presionar Guardar. 3. **Leer:** buscar el producto en Productos y en Inventario. 4. **Actualizar:** Editar → cambiar el margen a 60 % → Guardar. 5. **Validaciones:** intentar guardar sin nombre; intentar crear otro producto con el mismo código de barras. 6. **Eliminar:** presionar el ícono de baja, revisar el mensaje de confirmación y confirmar. 7. Después de cada operación, consultar la tabla producto en PostgreSQL.
Resultado esperado	• Crear: fila nueva en producto con precio sugerido $ 1.200 (800 × 1,50) y stock 20. • Leer: el producto aparece en Productos y en Inventario con stock 20 y estado Normal. • Actualizar: margen 60 % y precio de venta $ 1.280 en la base. • Validaciones: «El nombre es obligatorio» y «Ya existe un producto con ese código»; sin registros duplicados. • Eliminar: el sistema pide confirmación; luego el producto queda activo = false (baja lógica, se conserva el historial) y deja de listarse.
Resultado obtenido	• Crear: id_producto 13, precio_venta 1200,00, stock 20,000, activo = true. • Leer: el producto se muestra en ambas pantallas (figura 6). • Actualizar: porcentaje_margen 60,00 y precio_venta 1280,00 (figura 7). • Validaciones: ambos mensajes se mostraron (figura 8); la consulta por código devolvió 1 registro. • Eliminar: apareció la confirmación (figura 9); la fila quedó con activo = false y el producto dejó de aparecer en la búsqueda. **Resultado: correcta.**

Consultas a PostgreSQL después de cada operación (registro de ejecución):

```
[CREATE] fila en PostgreSQL:
id |           nombre            | stock  | costo  | margen | precio  | activo
----+-----------------------------+--------+--------+--------+---------+--------
 13 | Alfajor de maicena sin TACC | 20.000 | 800.00 |  50.00 | 1200.00 | t
(1 row)

[READ] el producto se visualiza en Productos e Inventario (figura 6).

[UPDATE] margen 50 → 60 (precio sugerido 800 × 1,60). Fila en PostgreSQL:
id |           nombre            | costo  | margen | precio
----+-----------------------------+--------+--------+---------
 13 | Alfajor de maicena sin TACC | 800.00 |  60.00 | 1280.00
(1 row)

[VALIDACIÓN] código duplicado rechazado. Registros con ese código en PostgreSQL:
cantidad
----------
         1
(1 row)

[DELETE] baja lógica confirmada. Fila en PostgreSQL (el historial se conserva):
id |           nombre            | activo
----+-----------------------------+--------
 13 | Alfajor de maicena sin TACC | f
(1 row)
```

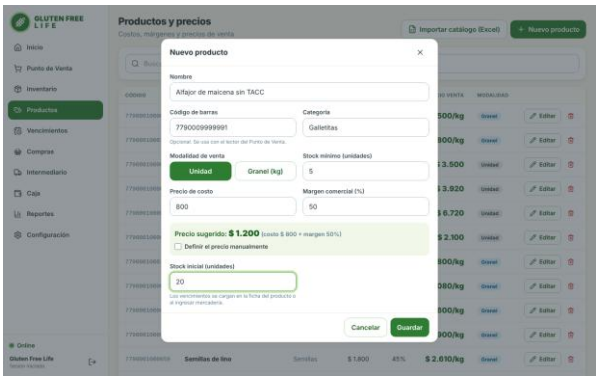


Figura 4. Crear: formulario completo con precio sugerido.

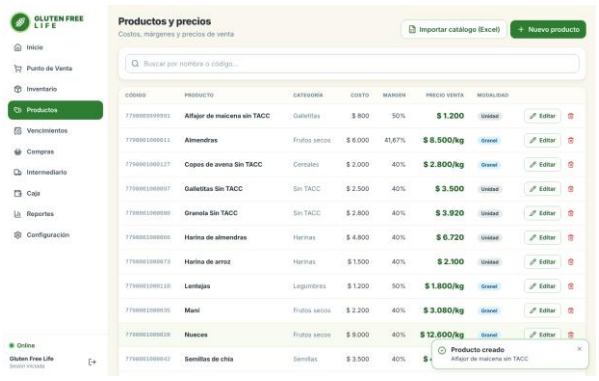


Figura 5. Crear: aviso «Producto creado» y alta en la lista.

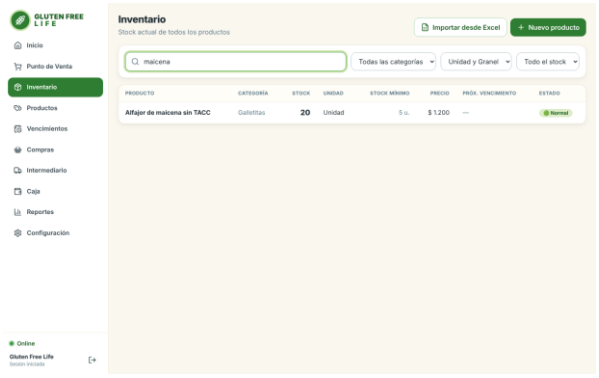


Figura 6. Leer: el producto se visualiza en Inventario (stock 20).

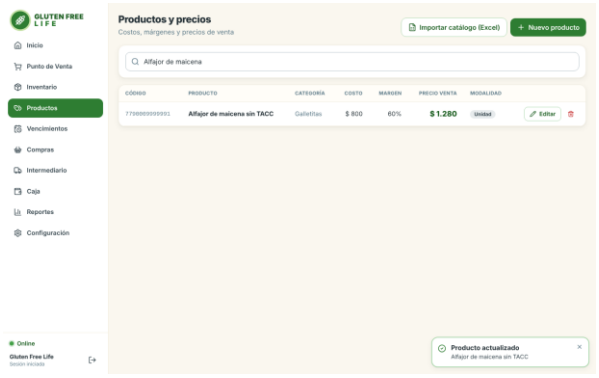


Figura 7. Actualizar: precio $ 1.280 y aviso «Producto actualizado».

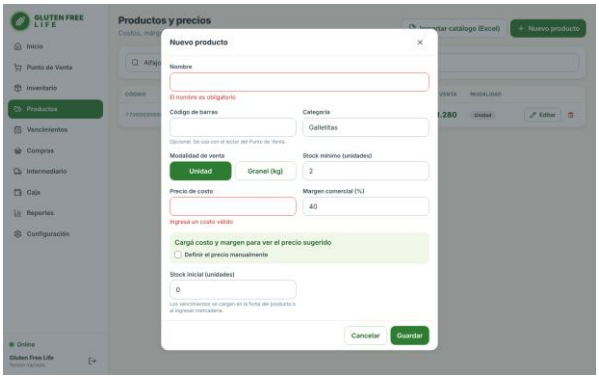


Figura 8a. Validación: campo obligatorio vacío.

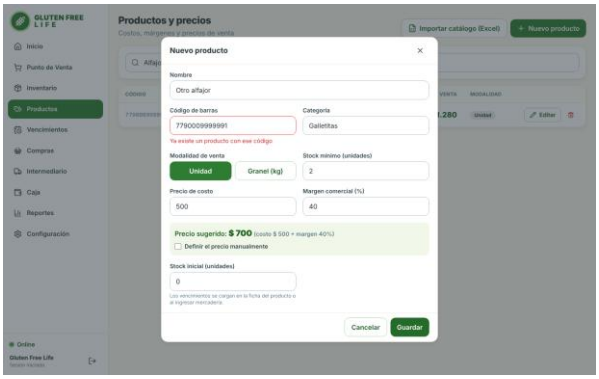


Figura 8b. Validación: código de barras duplicado.

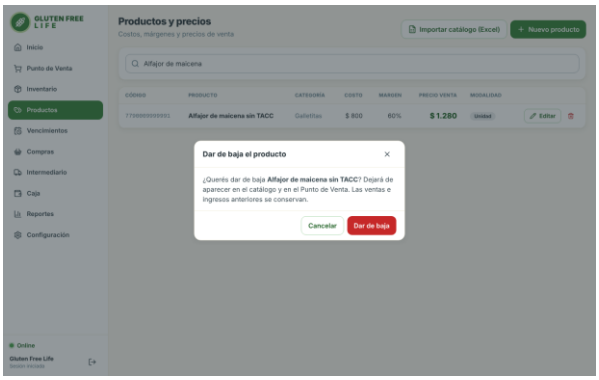


Figura 9. Eliminar: confirmación previa a la baja.

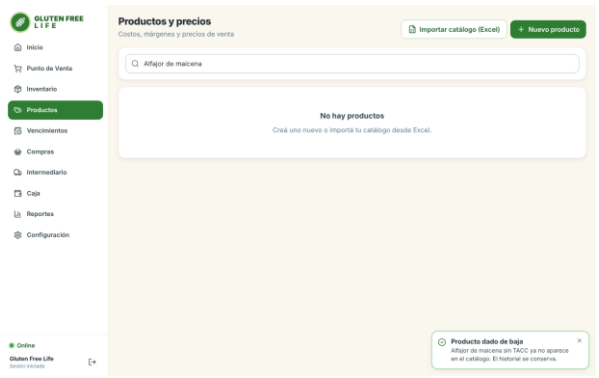


Figura 10. Eliminar: aviso «Producto dado de baja».

6.4 PI-3 · Suite de integración de la API

Además de los dos escenarios anteriores, el proyecto incluye una prueba de integración automática (backend/src/test/e2e.ts) que levanta la aplicación en un puerto dinámico, recarga los datos de

demostración y recorre la API contra PostgreSQL real. Se ejecuta con npm test (atención: recarga los datos de demostración, por lo que no debe usarse sobre una base con datos reales).

N.º	Grupo de pruebas	Reglas / funciones
1	health y login	Autenticación JWT
2	catálogo de demostración	Lectura de productos
3	venta: descuenta stock y el vencimiento más próximo; vuelto	R3, R4
4	renglón repetido: se unifica y pasa al vencimiento siguiente	R3, R4
5	validaciones de venta	Zod, reglas de dominio
6	sincronización offline: idempotente; stock insuficiente deja 0 y avisa	Modo offline
7	anular venta devuelve el stock	R12
8	vencimientos: filtros, estados y retiro	R6, R7
9	consolidar faltantes en un pedido al intermediario único	R8
10	ciclo de vida estricto del pedido; PDF y Excel; edición bloqueada	R9
11	ingreso de mercadería desde un pedido entregado	R10
12	duplicar pedido	Pedidos
13	caja: resumen, arqueo, cierre único y anulación bloqueada	R11, R12
14	reportes y dashboard	Consultas agregadas
15	importación de catálogo con errores	Validación fila por fila
16	productos: alta, precio sugerido, baja lógica y configuración	R2, CRUD
17	intermediario único, perfil y copia de seguridad	Configuración

Resultado obtenido: 17 grupos de pruebas correctos.

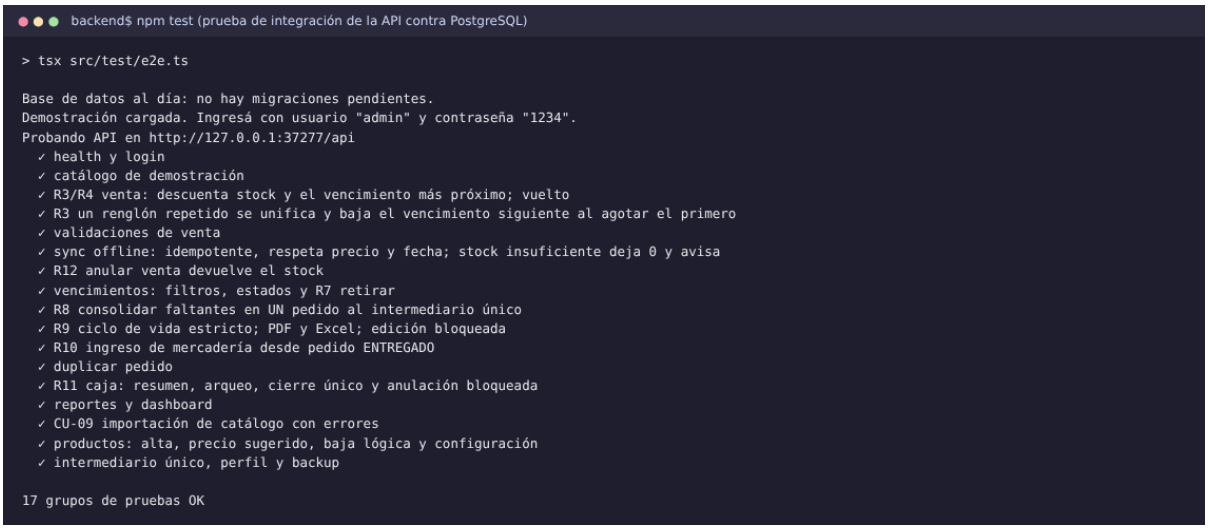


Figura 11. Ejecución de npm test: los 17 grupos de la prueba de integración de la API, correctos.

6.5 Defectos detectados durante las pruebas

Las propias pruebas permitieron encontrar y corregir dos defectos antes de la entrega:

N.º	Defecto	Corrección
1	La pantalla Configuración, abierta sin conexión, seguía mostrando «No se pudo cargar la información» aun después de restablecerse la conexión. Detectado en el recorrido de interfaz al capturar la sincronización offline.	La pantalla reintenta la carga automáticamente cuando vuelve la conexión (frontend/src/pages/Configuracion.tsx).
2	Los datos de demostración generaban las ventas «de hoy» restando minutos a la hora actual; de madrugada algunas caían en el día anterior y la prueba de caja (R11) fallaba. Detectado al ejecutar npm test a las 00:57.	El seed ubica las ventas dentro del día comercial local y no antes de su inicio (backend/src/seed.ts). Tras la corrección, los 17 grupos pasan a cualquier hora.

7. Alcance lógico de las pruebas

No se pretende probar la totalidad del sistema. Se eligieron pruebas representativas que demuestran el correcto funcionamiento de las funcionalidades principales del comercio y que son fáciles de repetir. Se priorizó la calidad, la relevancia y la claridad de los casos por sobre la cantidad.

7.1 Criterios de selección

- **Riesgo para el negocio:** se probó lo que, si falla, genera pérdidas o datos incorrectos: precios, stock, vencimientos, cobro y caja.
- **Reglas documentadas:** cada prueba se vincula con una regla de negocio del modelado (R1 a R12) o con una validación de datos.
- **Casos borde:** se incluyen los límites donde suelen aparecer errores (stock igual al mínimo, vencimiento que vence hoy, cantidad cero).
- **Repetibilidad:** todas se ejecutan con un comando y producen un resultado verificable.

7.2 Qué cubre cada tipo de prueba

Tipo	Parte del sistema cubierta	Justificación
Unitarias (25 casos)	Cálculo de precio (R2), stock crítico (R1), caducidad (R6), cantidad sugerida (R8), validación de venta e importación	Son las reglas más usadas y más propensas a errores sutiles; al ser funciones puras se prueban rápido y de forma aislada.
Integración PI-1	Interfaz de venta → API → servicio de ventas → PostgreSQL (stock, vencimiento más próximo, ticket, rechazo por stock)	La venta es la operación principal; combina varias reglas dentro de una transacción.
Integración PI-2	Pantalla de productos → API → validaciones → PostgreSQL (CRUD completo, duplicados, campos obligatorios, confirmación)	Demuestra Crear, Leer, Actualizar y Eliminar con persistencia real y mensajes claros.
Integración PI-3	API completa: pedidos de 7 estados, ingreso, caja, reportes, importación, sincronización offline, PDF y Excel	Verifica la coherencia de todos los módulos en conjunto.

7.3 Trazabilidad con las reglas de negocio

Regla	Descripción	Prueba unitaria	Prueba de integración
R1	Stock crítico: stock ≤ mínimo	PU-2	PI-1, PI-3
R2	Precio sugerido = costo × (1 + margen/100)	PU-1	PI-2, PI-3
R3	La venta es transaccional	—	PI-1, PI-3
R4	Descuento por el vencimiento más próximo	—	PI-1, PI-3
R6	Estados de vencimiento	PU-3	PI-3
R8	Cantidad sugerida de reposición	PU-4	PI-3
R9 a R12	Pedido de 7 estados, ingreso, caja, anulación	—	PI-3
Validaciones	Datos de venta, importación, campos obligatorios, duplicados	PU-5, PU-6	PI-2, PI-3

7.4 Límites del alcance

Para que la evaluación sea transparente, se indica lo que queda fuera de las pruebas automáticas:

- Las pantallas no cuentan con pruebas automáticas dentro del repositorio de la aplicación; el recorrido de interfaz de 26 pasos y la prueba PI-2 se ejecutan con Playwright mediante scripts incluidos en docs/pruebas.
- Las funciones que acceden a la base de datos (por ejemplo, reducirStock o descontarPorVencimiento) se prueban por integración y no de forma unitaria aislada.
- No se incluyeron pruebas de carga ni de seguridad. Mercado Pago y Tarjeta son medios de pago simulados, por lo que no se prueba ninguna pasarela real.
- La imagen de Docker no pudo construirse en el entorno de desarrollo; se recomienda ejecutar docker compose up --build una vez en el equipo de despliegue.

7.5 Cómo reproducir las pruebas

```
# Unitarias (no necesitan base de datos)
cd backend
npm install
npm run test:unit

# Integración de la API (recarga los datos de demostración)
DATABASE_URL=postgres://usuario:clave@localhost:5432/gfl npm test

# Unitarias + integración
DATABASE_URL=postgres://usuario:clave@localhost:5432/gfl npm run test:all

# Escenarios PI-1 y PI-2 (sistema en http://localhost:4000)
DATABASE_URL=... bash docs/pruebas/integracion-venta.sh
DATABASE URL=... CHROME=/ruta/a/chrome node docs/pruebas/ui-crud-productos.mjs
```